

Product Requirements Document (PRD): BDayPing (WhatsApp Reminder Bot)

1. Project Overview

Objective: Build a zero-cost, automated WhatsApp bot that stores family and friend birthdays and sends a reminder to the owner's WhatsApp number 2 hours before the birthday (e.g., at 10:00 PM the night before).
Core Interaction: The user adds birthdays via direct WhatsApp messages to their own number using text commands (e.g., `!add Aman 19-09`).
Hosting Strategy: Deployable on Render's Free Web Service tier (512MB RAM cap). Due to Render's ephemeral filesystem and 15-minute sleep cycle, the architecture requires an explicit "keep-alive" web server and remote database state persistence for both application data and WhatsApp authentication keys.

2. Technical Stack & Dependencies

The codebase must be built using Node.js with the following specific dependencies to bypass free-tier hardware limits:

- **WhatsApp Automation:** `@whiskeysockets/baileys` (Uses WebSockets instead of a headless Chromium browser, consuming ~50MB RAM compared to `whatsapp-web.js`'s ~500MB, preventing OOM crashes on Render).
- **Database:** `mongodb` (Connected to a free MongoDB Atlas cluster).
- **Auth Persistence:** `mongo-baileys` (Crucial: Render wipes local storage on every deploy. This package saves the Baileys session state directly to MongoDB so the user does not have to re-scan the QR code every 24 hours).
- **Scheduling:** `node-cron` (For the daily 10 PM check).
- **Web Server / Keep-Alive:** `express` (To bind to Render's required PORT and provide a `/ping` route for external uptime monitors).
- **QR Generation:** `qrcode-terminal` (To print the initial login code to the Render console).

3. Core Features & Functional Requirements

3.1. Express "Keep-Alive" Server

- Initialize a lightweight Express app listening on `process.env.PORT || 3000`.
- Expose a `GET /ping` route that returns `HTTP 200 "Bot is awake!"`.
- *Requirement Context:* The user will configure UptimeRobot to ping this route every 14 minutes to prevent Render from sleeping the container.

3.2. WhatsApp Authentication & Connection

- Connect to MongoDB Atlas using `process.env.MONGO_URI`.
- Use `mongo-baileys` (`useMongoDBAuthState`) to store and retrieve the authentication credentials from a MongoDB collection named `auth_sessions`.
- Initialize the Baileys socket (`makeWASocket`) using the MongoDB state.
- Log the QR code to the terminal if unauthenticated.
- Implement automatic reconnection logic if the socket drops (standard Baileys `connection.update` handling for `DisconnectReason`).

3.3. Command Listener (Data Ingestion)

- Listen to `messages.upsert` events.
- **Security Check:** Only process messages where `msg.key.fromMe === true` (messages the user sends to themselves/their own number).
- **Add Command:**
 - Trigger: `!add [Name] [DD-MM]` (e.g., `!add Aman 19-09`).
 - Action: Parse the input, validate the date format (DD-MM), and insert it into a MongoDB collection named `birthdays`.
 - Response: Send a confirmation WhatsApp reply (e.g.,  Added Aman on 19-09).
- **List Command:**
 - Trigger: `!list`
 - Action: Fetch all documents from the `birthdays` collection and format them into a readable text block.
 - Response: Send the formatted list as a WhatsApp reply.

3.4. Cron Job & Notification Engine

- Initialize a `node-cron` job set to run daily at 10:00 PM (`'0 22 * * *'`).
- Set the cron job timezone explicitly (e.g., `timezone: 'Asia/Kolkata'`) based on an environment variable `TZ`.
- **Logic:**
 1. Calculate tomorrow's date.
 2. Format it to `DD-MM`.
 3. Query the `birthdays` MongoDB collection for any records matching tomorrow's date.
 4. Loop through the matches and send a message to the user's own JID (WhatsApp ID).
 5. Message format:  Reminder: It's [Name]'s birthday tomorrow!

4. Environment Variables Required

The generated code must utilize the following environment variables:

- **PORT**: Port for the Express server.
- **MONGO_URI**: The MongoDB connection string.
- **OWNER_JID**: The user's WhatsApp ID (e.g., **919876543210@s.whatsapp.net**).
- **TZ**: Timezone for the cron job (e.g., **Asia/Kolkata**).

5. File Structure to Generate

The AI should output the complete code across these files:

- **package.json**: Containing all required scripts (**npm start**) and exact dependency versions.
- **index.js**: The main entry point containing the Express server, DB connection, Baileys initialization, and cron job setup.
- **Dockerfile** (Optional/Fallback): A standard Node.js Dockerfile (e.g., **FROM node:18-alpine**) mapping the port and starting the app, used if the user deploys to Hugging Face Spaces instead of Render.